ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

MERGING BERT EXPERTS WITH WEIGHT-ENSEMBLING MOE

Few-shot router training across five GLUE tasks

Group 1
Nguyễn Bá Anh – 20261218M
Nguyễn Thị Nhã Linh – 20261262M
Instructor: Nguyễn Kiêm Hiếu

ONE LOVE. ONE FUTURE.

Motivation: one model, many task experts

- Fine-tuned experts retain task-specific performance, but five BERT-base checkpoints store **547.4M parameters**.
- Static merging is compact (Task Arithmetic: **57.4** average score) but loses much of the experts' performance.
- **Question:** can input-conditioned MLP weights recover task performance with one merged model, at lower storage than five experts?

Scope

This is a BERT/GLUE adaptation of WEMoE, not a direct reproduction of the paper's CLIP/image experiments.[1]

HUST

[1] Anke Tang et al. "Merging Multi-Task Models via Weight-Ensembling..." (2024).

Theory: route in task-vector space

For expert e, define its task vector relative to the pretrained weights:

$$\tau_e = W_e - W_0.$$

WEMoE constructs an input-conditioned MLP weight:

$$W(x) = W_0 + \sum_{e=1}^{E} w_e(x)\, \tau_e.$$

- W_0 carries shared pretrained knowledge; τ_e captures an expert's task-specific change.
- The router weights $w_e(x)$ are **unnormalised**: they may amplify or subtract task vectors.
- This project adapts the weight-ensembling idea to BERT MLPs.[1]

[1] Anke Tang et al. "Merging Multi-Task Models via Weight-Ensembling..." (2024).

Method: BERT MLPs become input-conditioned

- Backbone: `bert-base-uncased`; experts fine-tuned separately for MRPC, RTE, CoLA, SST-2, and QNLI.
- For each BERT layer, a router uses the mean token representation over non-padding tokens to produce expert weights.
- Full scope ensembles both `intermediate.dense` and `output.dense`; intermediate-only ensembles the former.
- Other weights use Task Arithmetic ($\lambda = 0.3$); task-specific classification heads are retained.
- FSFT trains only the routers (7.1M parameters); the backbone and the task experts stay frozen.

task experts $\longrightarrow \{\tau_e\} \longrightarrow$ input router $w(x) \longrightarrow W(x)$ in each selected MLP

Data boundary: reserve router examples, select on dev

Portion of each official GLUE train split	Size	Used for
`expert_train`	Remaining examples	Fine-tune the task experts
`fewshot_pool`	256 per task	FSFT router training only
`dev`	10%, at most 1,000	Choose expert epoch, λ, and router step
Official validation	Task-specific	Final reported metrics only

Leakage control

Experts never train on the few-shot pool or dev split; official validation is not used for model selection.

Experiments: compare merging, routing, and expert limits

- Five tasks: MRPC, RTE, CoLA, SST-2, QNLI; BERT-base with max sequence length 128.
- Baselines: majority prediction, simple averaging, Task Arithmetic (λ chosen on dev), and separately fine-tuned experts.
- WEMoE uses $\lambda = 0.3$ for its non-MLP merge and router initialization; FSFT trains the routers with labeled examples, selecting the best dev checkpoint every 50 steps (up to 500).

Task score

MRPC: mean of accuracy and F1; CoLA: Matthews correlation (MCC); RTE, SST-2, QNLI: accuracy. Scores below are scaled by 100.

Main result: FSFT recovers much of the merging gap

Model	Average task score
Majority class	45.8
Simple Averaging ($\lambda = 1/5$)	53.3
Task Arithmetic ($\lambda = 0.3$)	57.4
WEMoE init (before FSFT)	57.8 ± 0.6
WEMoE + FSFT, full MLP ($k = 256$)	$\mathbf{69.9} \pm \mathbf{0.1}$
Five single-task experts	78.2

- Gain over majority, (score − majority)/(expert − majority): Task Arithmetic **18%**, WEMoE + FSFT **59%**; Task Arithmetic predicts one class on MRPC and RTE.
- Router initialisation alone does not help (57.8): the gain comes from FSFT.
- Reported means and standard deviations are over router seeds 0, 1, and 2; they are not recomputed from the partial local raw runs.

Few-shot budget: $k = 256$ is stable, small k is not

k per task	Avg score	Gain over majority	CoLA MCC
16	60.8 ± 2.7	25.5%	27.6 ± 14.1
64	64.6 ± 3.7	46.7%	20.5 ± 17.8
256	$\mathbf{69.9 \pm 0.1}$	59.1%	42.6 ± 0.9

- Full-MLP WEMoE, reported means $\pm$ std over router seeds 0, 1, 2.
- Small budgets overfit early (best dev step 50–250 of 500) and vary strongly across seeds.
- The average moves 3.8 points from $k = 16$ to 64, but gain over majority almost doubles: part of the $k = 16$ average is MRPC/RTE at their constant-guess level.

Per-task results: aggregate gains hide a task trade-off

	MRPC acc/F1	RTE acc	CoLA MCC	SST-2 acc	QNLI acc
Majority	68.4/81.2	52.7	0.0	50.9	50.5
Full MLP, $k = 256$	76.3 ± 4.5 / 83.9 ± 2.8	52.0 ± 4.4	42.6 ± 0.9	88.0 ± 0.5	86.7 ± 0.7
Intermediate, $k = 256$	66.5 ± 0.7 / 76.6 ± 1.0	59.0 ± 0.9	41.7 ± 0.9	86.7 ± 0.2	86.5 ± 0.2

Do not read the mean as universal dominance

Full-MLP WEMoE is below the RTE majority baseline (52.0 vs. 52.7). Intermediate-only improves RTE but is below the MRPC majority (66.5/76.6 vs. 68.4/81.2).

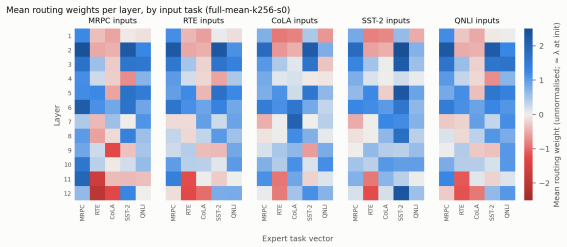
RTE (277) and MRPC (408) validation sets are small (standard error about 3 and 2 points), so part of the full vs. intermediate gap may be noise.

Storage trade-off: intermediate-only is smaller, not uniformly better

Deployment	Stored params	vs. five models	Avg score	Task contrast
Five experts	547.4M	100%	78.2	Strongest average
WEMoE, full MLP	400.0M	73.1%	69.9 ± 0.1	MRPC 76.3; RTE 52.0
WEMoE, intermediate	258.4M	47.2%	69.1 ± 0.1	MRPC 66.5; RTE 59.0

- Intermediate-only stores about **35% fewer parameters** than full-MLP WEMoE, at a similar reported average score.
- **Caveat:** each task keeps its own head, so the task is known at inference; five experts are then more accurate and only 37% larger than full-MLP WEMoE. The storage case rests on the intermediate-only variant or on settings with unknown task.
- Intermediate-only reaches its best step late (400–500 of 500), so it may improve with more steps.
- Reported FSFT training estimates on one Tesla T4: about 14–16 minutes per full run vs. 8.5 minutes intermediate; hardware-specific, not inference latency.

Routing: shared structure, and why full-MLP fails on RTE



- Similar patterns across input tasks: mean-matrix correlation 0.82; input task explains about 58% of routing variance. Weights span -3.0 to 4.4 (unnormalised).
- **RTE:** the full-MLP router gives the RTE task vector weight ≈ 0 even on RTE inputs (own expert: 0.01, vs. 1.37 for MRPC) and subtracts it in layers 11–12 (-1.2). Intermediate-only gives it 0.96 on average and reaches 59.0.

Report figure: full MLP, $k = 256$, seed 0. The input-independent routing ablation is pending; this plot alone does not establish that input dependence causes the score gains.

Limitations: encouraging aggregate, unresolved failures

- **RTE remains a failure case:** full MLP scores 52.0, below the 52.7 majority baseline; the underlying RTE expert is also relatively weak.
- **Small budgets are variable:** at $k = 16$, CoLA MCC standard deviation is 14.1 across router seeds.
- **Selection sees more labels:** dev uses up to 1,000 examples per task, even when FSFT trains on only $k = 256$.
- **Incomplete evidence:** experts/data split use one seed; local raw router results do not contain all seeds behind the report aggregates.
- **Efficiency gap:** inference latency and the input-independent ablation remain pending.

The paper uses CLIP/image tasks and unlabeled test-time entropy minimisation for the router; here FSFT uses 256 labelled examples per task. Comparisons are qualitative.[1]

HUST

[1] Anke Tang et al. "Merging Multi-Task Models via Weight-Ensembling..." (2024).

Conclusion: WEMoE helps, but the task-level trade-offs matter

Answer

In this first full BERT/GLUE run, few-shot router training raises the reported average to 69.9, recovering much of the gap left by static merging.

- Full MLP and intermediate-only WEMoE reach similar averages with different storage and per-task outcomes.
- RTE remains below majority for full MLP; do not infer per-task dominance from an aggregate.
- More complete seeds, stricter few-shot model selection, the routing ablation, and measured inference latency are needed next.

Backup: task metrics and aggregation

- **MRPC:** task score is the mean of accuracy and F1; the per-task table reports both values.
- **CoLA:** task score is Matthews correlation coefficient (MCC), not accuracy.
- **RTE, SST-2, QNLI:** task score is accuracy.
- **Average score:** unweighted mean of the five task scores; displayed on a $\times 100$ scale.
- **Uncertainty:** WEMoE results are reported as mean $\pm$ standard deviation over router seeds 0, 1, and 2; experts and data split are fixed.

Backup: router training details

- The shared data split reserves 256 labeled examples per task for FSFT; experts never see those examples.
- Routers use two hidden layers, mean-pooled non-padding-token inputs, learning rate 10^{-4}, and up to 500 update steps.
- The report counts 7.1M trainable router parameters; the pretrained backbone and task experts are not fine-tuned during FSFT.
- Best step is selected using dev evaluations every 50 steps; dev contains up to 1,000 examples per task.
- Reported figures are from the project's first full run, not a general performance guarantee or a complete reproduction of WEMoE.

References

[1] Anke Tang et al. *Merging Multi-Task Models via Weight-Ensembling Mixture of Experts*. 2024. arXiv: 2402.00433 [cs.LG].